Linux TIMER
开发指南

版本号: 1.5

发布日期: 2025.9.24

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2022.08.04	XAA0249	创建初版
1.1	2024.03.20	XAA0312	更新适用范围
1.2	2024.10.23	XAA0309	增加 Linux-5.4 内核版本支持，并修改非 bsp 独立仓库配置。
1.3	2025.3.7	XAA0331	增加 Linux-6.6 内核版本支持。
1.4	2025.8.9	XAA0368	修改格式文档。
1.5	2025.9.24	XAA0339	补充支持的 Linux 版本。

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 引言 1
1.1 文档简介 1
1.2 目标读者 1
1.3 适用范围 1
1.4 文档约定 1
1.4.1 标志说明 1
1.4.2 地址与数据描述方法约定 2
1.4.3 数值单位约定 2
1.5 相关术语介绍 2

2 模块描述 4
2.1 模块功能 4
2.2 模块配置 4
2.2.1 dts 配置说明 4
2.2.2 menuconfig 配置说明 5
2.2.2.1 bsp SDK 5
2.2.2.2 tina SDK 7
2.3 源码结构介绍 7
2.4 软件框架 7

3 模块设计 8
3.1 关键数据定义 8
3.2 关键数据结构 8
3.3 模块流程设计 9
3.3.1 初始化流程 9
3.3.2 中断处理流程 9
3.3.3 休眠/唤醒处理流程 9

4 模块接口说明 10
4.1 内核态接口 10
4.1.1 sun50i_clkevt_time_setup 10
4.1.2 sun50i_clkevt_time_start 10
4.1.3 sun50i_clkevt_time_stop 10
4.2 用户态接口 11
4.2.1 time 11
4.2.2 gettimeofday 11
4.2.3 settimeofday 11
4.2.4 adjtime 12

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 功能开发 13
5.1 内核态定时器开发 13
5.1.1 内核态定时器开发流程 13
5.1.2 内核态定时器开发示例代码 13
5.2 用户态定时开发 14
5.2.1 用户态定时器开发流程 14
5.2.2 用户态定时器开发示例代码 14

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 引言

1.1 文档简介

介绍 Linux 内核中 Timer 驱动的适配和使用方法，为 Timer 设备的使用者和维护者提供参考。

1.2 目标读者

Timer 设备的使用者和维护者

1.3 适用范围

适用于 allwinnertech 全系列平台。

linux	5.4、5.10、5.15、6.1、6.6、6.6-xuantie	
硬件平台	allwinner 全平台	

1.4 文档约定

1.4.1 标志说明

⚠ 警告

- 提醒操作中应注意的事项。不当的操作可能会损坏器件，影响可靠性、降低性能等。

注意

为准确理解文中指令、正确实施操作而提供的补充或强调信息。

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

技巧

一些容易忽视的小功能、技巧。了解这些功能或技巧能帮助解决特定问题或者节省操作时间。

1.4.2 地址与数据描述方法约定

本文档在描述地址、数据时遵循如下约定：

表 1-2: 地址与数据描述方法约定

符号	例子	说明
0x	0x0200，0x79	地址或数据以 16 进制表示。
0b	0b010，0b00 000 111	数据采用二进制表示 (寄存器描述除外)。
X	00X，XX1	数据描述中，X 代表 0 或 1。
		例如，00X 代表 000 或 001；XX1 代表 001，011，101 或 111。

1.4.3 数值单位约定

本文档在描述数据容量（如 NAND 容量）时，单位词头代表的是 1024 的倍数；描述频率、数据速率等时则代表的是 1000 的倍数。具体如下：

表 1-3: 数值单位约定

类型	符号	对应数值
数据容量（如 NAND 容量）	1 K	1024
	1 M	1 048 576
	1 G	1 073 741 824
频率，数据速率等	1 k	1000
	1 M	1 000 000
	1 G	1 000 000 000

1.5 相关术语介绍

版权所有 © 珠海全志科技股份有限公司。保留一切权利

序号	术语或缩略	解释说明
1	Sunxi	指 Allwinner 的一系列 SOC 硬件平台。
2	Timer	内核定时器，到规定数值后，产生中断标志 Timer 任务的完成

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 模块描述

2.1 模块功能

Timer（定时器/计数器）以一定的频率计数；到规定数值后，产生中断标志 Timer 任务的完成。（如：Timer 为子系统产生内部中断，实现精准有效的系统时间管理。）

2.2 模块配置

2.2.1 dts 配置说明

在不同的 Sunxi 硬件平台中，Timer 控制器的数目也不同，但对于每一个 Timer 控制器来说，在 dtsi 中配置参数相似，如下：

```
soc_timer0: timer@3009000 {
    compatible = "allwinner,hstimer-v100";
    device_type = "soc_timer";
    reg = <0x0 0x03009000 0x0 0x400>;
    interrupt-parent = <&gic>;
    interrupts = <GIC_SPI 89 IRQ_TYPE_LEVEL_HIGH>;
    clock-names = "parent", "bus", "timer0-mod", "timer1-mod";
    clocks = <&sys24M>, <&ccu CLK_BUS_TIMER>, <&ccu CLK_TIMER0>, <&ccu CLK_TIMER1>;
    resets = <&ccu RST_BUS_TIMER0>;
};
```

其中：

1. compatible: 表征具体的设备, 用于驱动和设备的绑定。
2. device_type：表征是 soc 级 Timer。
3. reg: 设备使用的地址。
4. interrupt-parent: 设备引用的中断节点。
5. interrupts: 设备使用的中断号。
6. clocks: 设备使用的时钟。
7. resets: 设备使用的复位时钟。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.2.2 menuconfig 配置说明

2.2.2.1 bsp SDK

在命令行中进入 SDK 根目录，执行./build.sh menuconfig 进入配置主界面，并按以下步骤操作。首先，选择 Allwinner BSP 选项进入下一级配置，如下图所示：

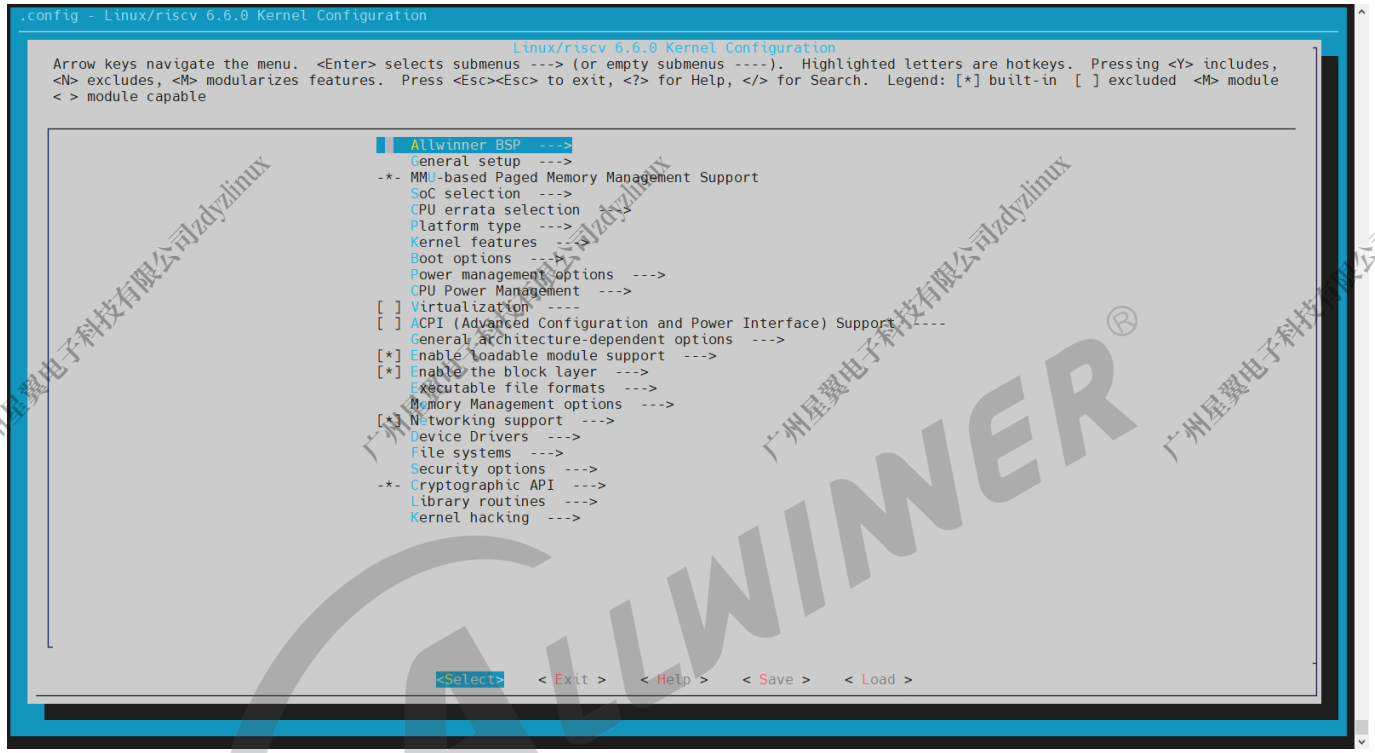

图 2-1: Device-Drivers 配置

然后，选择 Device Drivers 选项，进入下一级配置，如下图所示：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
.config - Linux/riscv 6.6.0 Kernel Configuration
> Allwinner BSP
                                    Allwinner BSP
  Arrow keys navigate the menu.  <Enter> selects submenus ---> (or empty submenus ----).  Highlighted letters are hotkeys.  Pressing <Y> includes,
  <N> excludes, <M> modularizes features.  Press <Esc><Esc> to exit, <?> for Help, </> for Search.  Legend: [*] built-in  [ ] excluded  <M> module
  < > module capable

                        Platform Information  --->
                        Allwinner's Debugging Options  --->
                        Allwinner's Turbo Options  --->
                        Device Drivers  --->
                        Library routines  --->
                        NAND Drivers  ----
                        GPU Drivers  --->

                        <Select>    < Exit >    < Help >    < Save >    < Load >
```

图 2-2: clock-source 配置

接着选择 Timer Drivers 选项, 进入下一级配置, 如图 4 所示:

```
.config - Linux/riscv 6.6.0 Kernel Configuration
> Allwinner BSP > Device Drivers
                                    Device Drivers
  Arrow keys navigate the menu.  <Enter> selects submenus ---> (or empty submenus ----).  Highlighted letters are hotkeys.  Pressing <Y> includes,
  <N> excludes, <M> modularizes features.  Press <Esc><Esc> to exit, <?> for Help, </> for Search.  Legend: [*] built-in  [ ] excluded  <M> module
  < > module capable

                        Clock Drivers  --->
                        Pinctrl Drivers  --->
                        UART Drivers  --->
                        Timer Drivers  --->
                        DMA Drivers  --->
                        RTC Drivers  --->
                        Bus Drivers  --->
                        Dump-Reg Drivers  --->
                        Watchdog Drivers  --->
                        IOMMU Drivers  --->
                        MBUS Drivers  --->
                        SD/MMC Drivers  ----
                        SUNXI UFS Drivers  ----
                        NAND Drivers  ----
                    <*> Memory Technology Device (AW_MTD) support  --->
                    < > Sunxi redeposit startup IO data
                        Video Drivers  --->
                        G2D Drivers  --->
                        DI_Drivers  --->
                        VIN (camera) Drivers  --->
                        VFE (camera) Drivers  --->
                        TVD (cvbs in) Drivers  ----
                        DRM Drivers  ----
                        Sunxi Fast Car Reverse Drivers  ----
                        Sunxi Fast Car Reverse Drivers For Amp  ----
                        TVIN Drivers  --->
                        VE Drivers  --->
                        rt-media  --->
                        SOUND Drivers  ----
                        *** Choose one according to the actual usage ***
                        CE Drivers  --->
                    ⊥(+)

                        <Select>    < Exit >    < Help >    < Save >    < Load >
```

图 2-3: timer 配置选项

具体是选择 sun4i 还是 sun50i 需要结合 ic 设计决定。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.2.2.2 tina SDK

首先运行 source build/envsetup.sh。

第二步运行 lunch 命令选择具体方案。

第三步运行 make kernel_menuconfig 命令进入配置页面，参考 BSP SDK 的配置主页面步骤完成配置。

2.3 源码结构介绍

Timer 驱动的源代码位于内核在 SDK/bsp/drivers/timer 目录下：

```
SDK/bsp/drivers/timer

 ├── Makefile
 ├── Kconfig
 ├── timer-sun50i.c      // 使用sun5i 平台的 Timer 控制器驱动代码
 ├── timer-sun4i.c      // 使用suni 平台的 Timer 控制器驱动代码
```

2.4 软件框架

Timer 驱动模块内部的功能可划分如下所示：

1. 驱动注册，向 Core 注册 Timer 驱动，驱动注册后会跟 Device Tree 中描述的 Timer 控制器进行匹配，匹配成功后会调用 init 函数，初始化 Timer 控制器。

2.Timer 操作集，提供一套标准的接口集，内核可以通过这些接口控制 Timer 控制器完成计时功能。

3. 中断处理，Timer 控制器在计时结束时会触发中断。

4. 休眠处理，在 Timer 被 shutdown 时会保存寄存器。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 模块设计

3.1 关键数据定义

```
#define TIMER_SYNC_TICKS    3
```

TIMER_SYNC_TICKS 用于同步时钟

```
static u32 regs_backup[ARRAY_SIZE(regs_offset)];
```

用于在 shutdown 时保存寄存器信息

3.2 关键数据结构

regs_offset 结构体

```
/* Registers which needs to be saved and restored before and after sleeping */
static u32 regs_offset[] = {
    TIMER_IRQ_REG,
    TIMER_STA_REG,
    TIMER_CTL_REG(0),
    TIMER_IVL_REG(0),
    TIMER_CVL_REG(0),
    TIMER_IVH_REG(0),
    TIMER_CVH_REG(0),
    TIMER_CTL_REG(1),
    TIMER_IVL_REG(1),
    TIMER_CVL_REG(1),
    TIMER_IVH_REG(1),
    TIMER_CVH_REG(1),
};
```

休眠唤醒时需要保存的寄存器信息

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.3 模块流程设计

3.3.1 初始化流程

```
flowchat
st=>start: 开始
e=>end: 结束
op=>operation: timer of init
op1=>operation: 获取并设置时钟
op2=>operation: 注册timer
op3=>operation: 注册中断

st->op->op1->op2->op3->e
```

3.3.2 中断处理流程

```
flowchat
st=>start: 开始
e=>end: 结束
op=>operation: 清除中断标志位
op1=>operation: 执行中断注册的回调函数


st->op->op1->e
```

3.3.3 休眠/唤醒处理流程

```
flowchat
st=>start: 开始
e=>end: 结束
op=>operation: 获取timer_of
op1=>operation: 保存指定的寄存器信息
op2=>operation: 停止timer计时

st->op->op1->op2->e
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 模块接口说明

4.1 内核态接口

4.1.1 sun50i_clkevt_time_setup

- 函数原型：void sun50i_clkevt_time_setup(void __iomem *base, u8 timer, unsigned long delay)
- 作用：Timer 参数设置。
- 参数：
  - 参数 1:Timer 基地址。
  - 参数 2:Timer 号。
  - 参数 3: 延迟时长。
- 返回：无返回值。

4.1.2 sun50i_clkevt_time_start

- 函数原型：void sun50i_clkevt_time_start(void __iomem *base, u8 timer, bool periodic)
- 作用：开始 Timer 计时。
- 参数：
  - 参数 1:Timer 基地址。
  - 参数 2:Timer 号。
  - 参数 3: 是否周期计数。
- 返回：无返回值。

4.1.3 sun50i_clkevt_time_stop

- 函数原型：static void sun50i_clkevt_time_stop(void __iomem *base, u8 timer)
- 作用：停止 Timer 计时。
- 参数：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

  - 参数 1:Timer 基地址。
  - 参数 2:Timer 号。

- 返回：无返回值。

4.2 用户态接口

4.2.1 time

- 函数原型：time_t time(time_t *t)
- 作用：返回当前时间点到 Linux 纪元的秒数。
- 参数：
  - 参数 1:time_t 类型变量。
- 返回：
  - 大于 0: 成功返回当前时间点到 Linux 纪元的秒数。
  - -1: 失败。

4.2.2 gettimeofday

- 函数原型：int gettimeofday(struct timeval *tv, struct timezone* tz)
- 作用：获取当前时间，包括秒和微秒。
- 参数：
  - 参数 1：用于存储秒数和微妙。
  - 参数 2：用于存储时区。
- 返回：
  - 0: 成功。
  - -1: 失败。

4.2.3 settimeofday

- 函数原型：int settimeofday(const struct timeval *tv, const struct timezone* tz)
- 作用：设置当前时间戳。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 参数：
  - 参数 1：用于存储秒数和微妙。
  - 参数 2：用于存储时区。
- 返回：
  - 0: 成功。
  - -1: 失败。

4.2.4 adjtime

- 函数原型：int adjtime(const struct timeval *delta, struct timeval* olddelta)
- 作用：调整系统时间。
- 参数：
  - 参数 1：指定时间调整量。
  - 参数 2：返回实际调整量。
- 返回：
  - 0: 成功。
  - -1: 失败。

广州星翼电子科技有限公司zdyzlinux

5 功能开发

5.1 内核态定时器开发

5.1.1 内核态定时器开发流程

步骤 1 创建 hrtimer 定时器结构体

```
static struct hrtimer timer; /* 创建hrtimer定时器 */
```

步骤 2 初始化 hrimter 结构体

```
hrtimer_init(&timer, CLOCK_MONOTONIC, HRTIMER_MODE_REL_HARD);
```

注意

使用 *CLOCK_MONOTONIC* 时间，*HRTIMER_MODE_REL_HARD* 表示在硬中断环境下处理。

步骤 3 创建超时处理函数

```
timer.function = hrtimer_handler; /* 设置超时处理函数 */
```

注意

到设置的超时时间后，在超时处理函数进行超时后相应的操作。

步骤 4 取消定时器

```
hrtimer_cancel(&timer); /* 取消定时器 */
```

5.1.2 内核态定时器开发示例代码

在内核中直接使用 hrtimer 示例代码如下，每 8ms 周期性触发并打印 log。

```
static struct hrtimer timer; /* 创建hrtimer定时器 */

/* 定时器到期处理函数 */
static enum hrtimer_restart hrtimer_handler(struct hrtimer *hrt)
{
        printk("hrtimer_handler");
        hrtimer_forward_now(hrt, 8000000);  /* 将超时时间向后移8000000ns=8ms */
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
        return HRTIMER_RESTART;  /* 返回重新启动标志，无需再次调用hrtimer_start */
}

static int __init hrtimer_test_init(void)
{
        /* 初始化hrtimer，使用CLOCK_MONOTONIC时间，HRTIMER_MODE_REL_HARD表示在硬中断环境下处理 */
        hrtimer_init(&timer, CLOCK_MONOTONIC, HRTIMER_MODE_REL_HARD);
        timer.function = hrtimer_handler; /* 设置超时处理函数 */
        /* 启动定时器 */
        hrtimer_start(&timer, 8000000, HRTIMER_MODE_REL_HARD);
}

static void __exit hrtimer_test_exit(void)
{
        hrtimer_cancel(&timer);  /* 取消定时器 */
}
```

5.2 用户态定时开发

5.2.1 用户态定时器开发流程

步骤 1 创建 timer_t 时间戳变量

```
time_t timestamp;
```

步骤 2 创建 struct timeval 结构体变量用于存储时间

```
struct timeval tv;
```

步骤 3 调用标准接口设置或获取时间

```
settimeofday(&tv, NULL);

gettimeofday(&tv1, NULL);
```

5.2.2 用户态定时器开发示例代码

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <sys/time.h>

int main() {
    /*
    * 假设你有一个时间戳，这里以秒为单位
    * 注意：这里的时间戳应该是自1970年1月1日以来的秒数
    * 你可能需要将时间戳转换为struct timeval需要的微秒精度
    */
    time_t timestamp = 1609459200; /* 示例时间戳，比如2021年1月1日 */
    /* 转换为struct timeval，因为settimeofday需要这个结构体 */
    struct timeval tv;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    struct timeval tv1;
    tv.tv_sec = timestamp;
    tv.tv_usec = 0; /* 微秒部分设置为0 */
    /* 调用settimeofday设置系统时间 */
    if (settimeofday(&tv, NULL) == -1) {
        perror("settimeofday");
        exit(EXIT_FAILURE);
    }
    printf("System time set successfully\n");  /* 打印成功信息 */

    gettimeofday(&tv1, NULL); /* 获取当前时间 */
    printf("Seconds: %ld\n", tv1.tv_sec); /* 打印秒数 */
    printf("Microseconds: %ld\n", tv1.tv_usec); /* 打印微秒数 */
    printf("System time get successfully\n");  /* 打印成功信息 */

    return 0;
}
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。